Web Application Basics

Platform: TryHackMe
Category: Web Application Fundamentals
Room Link: https://tryhackme.com/room/webapplicationbasics

1. Objective

Understand how web applications work and how different components communicate with each other. Learn the structure of URLs, HTTP requests and responses, common request methods, status codes, and HTTP headers. The room also introduces important security headers and explains how they help protect web applications against common attacks.

2. Key Concepts Learned

- **Web Application:** An application that runs through a web browser and allows users to interact with online services.
- **Front End:** The visible part of a web application that users interact with.
- **Back End:** The server-side components responsible for processing requests, managing information, and supporting application functionality.
- **HTML (HyperText Markup Language):** Defines the structure and content displayed on a webpage.
- **CSS (Cascading Style Sheets):** Controls the appearance, colours, fonts, and layout of webpages.
- **JavaScript (JS):** Adds functionality and interactive behaviour to web applications.
- **Database:** Stores, modifies, and retrieves application information.
- **Infrastructure:** Includes web servers, application servers, storage, networking devices, and other supporting components.
- **WAF (Web Application Firewall):** Filters potentially dangerous incoming requests before they reach the web server.
- **URL (Uniform Resource Locator):** The web address used to access a particular resource on the internet.
- **URL Components:** Scheme, User, Host/Domain, Port, Path, Query String, and Fragment.
- **HTTP (HyperText Transfer Protocol):** A communication protocol that allows browsers and web servers to exchange information.
- **HTTPS:** An encrypted version of HTTP that protects communication between a browser and a web server.
- **HTTP Messages:** The requests sent by clients and the responses returned by servers.
- **HTTP Request Methods:** GET, POST, PUT, DELETE, PATCH, HEAD, OPTIONS, TRACE, and CONNECT.
- **HTTP Versions:** HTTP/0.9, HTTP/1.0, HTTP/1.1, HTTP/2, and HTTP/3.
- **HTTP Headers:** Key-value pairs that provide additional information about requests and responses.

- **Request Body Formats:** URL Encoded, Form Data, JSON, and XML.
- **HTTP Status Codes:** Three-digit codes that indicate the result of a client's request.
- **Security Headers:** Additional HTTP response headers that help protect web applications against common security threats.
- **Content Security Policy (CSP):** Defines trusted sources from which a browser can load scripts and other resources.
- **HTTP Strict Transport Security (HSTS):** Instructs browsers to communicate with a website using HTTPS.
- **X-Content-Type-Options:** Prevents browsers from guessing the MIME type of a resource.
- **Referrer-Policy:** Controls how much referrer information is shared when navigating between websites.

3. Tools and Websites Covered

- **TryHackMe:** Provided the learning material and practical environment for understanding web application fundamentals.
- **Web Browser:** Used to access and interact with web applications.
- **Static Site Emulator:** Used in the practical task to construct HTTP requests and observe server responses.
- **securityheaders.io:** Used as a reference website for analysing HTTP security headers.
- **Web Server:** Responsible for hosting and delivering web application content.
- **Database:** Stores and manages information required by web applications.

4. Practical Work

- Explored the basic architecture of web applications and understood how front-end and back-end components work together.
- Studied the roles of HTML, CSS, and JavaScript in displaying content and providing interactive functionality.
- Examined back-end components, including databases, web servers, infrastructure, and Web Application Firewalls.
- Broke down URL structures and identified the purpose of each component, including the scheme, domain, port, path, query string, and fragment.
- Studied HTTP messages and understood how clients send requests and servers return responses.
- Examined the structure of HTTP messages, including the start line, headers, empty line, and body.
- Compared HTTP request methods and learned how GET, POST, PUT, DELETE, and PATCH are used to interact with web resources.
- Reviewed other HTTP methods, including HEAD, OPTIONS, TRACE, and CONNECT.
- Explored the development of HTTP protocol versions, from HTTP/0.9 to HTTP/3, and understood improvements such as persistent connections, multiplexing, and header compression.

- Examined common request headers, including Host, User-Agent, Referer, Cookie, and Content-Type.
- Studied different request body formats and how applications exchange information through URL-encoded data, multipart form data, JSON, and XML.
- Reviewed HTTP response status codes and their five main categories, from informational responses to server errors.
- Examined response headers, including Date, Content-Type, Server, Set-Cookie, Cache-Control, and Location.
- Studied cookie security attributes and understood the purpose of HttpOnly and Secure flags.
- Explored important HTTP security headers, including CSP, HSTS, X-Content-Type-Options, and Referrer-Policy.
- Reviewed how CSP controls trusted content sources and how HSTS helps enforce secure HTTPS connections.
- Studied how the `nosniff` directive prevents browsers from interpreting resources as a different MIME type.
- Used the TryHackMe static-site emulator to understand how different HTTP methods can retrieve, update, and delete application data.

Practical HTTP Requests

The final task involves performing three operations through the TryHackMe HTTP request emulator.

1. GET Request Retrieve Users

```
GET /api/users
```

Used to retrieve the list of users available through the API.

2. POST Request Update User Information

```
POST /api/user/2
```

Used to update Bob's country from `UK` to `US`.

3. DELETE Request Remove a User

```
DELETE /api/user/1
```

Used to delete the user with ID `1`.

5. Important Security Observations

- **Front-end and back-end security both matter.** A web application can have a well-designed interface but still contain weaknesses in its server-side functionality.

- **URLs can expose sensitive information.** Login credentials and other confidential information should not be placed in URLs because they may appear in browser history or server logs.
- **Typosquatting can support phishing attacks.** Attackers register domain names that closely resemble legitimate websites to trick users into visiting malicious pages.
- **User-controlled input requires validation.** Query strings, URL paths, and request body parameters can be manipulated and should be handled securely.
- **HTTP methods can affect application data.** POST, PUT, PATCH, and DELETE operations should have proper authorization checks.
- **HTTP headers reveal useful information.** Request and response headers can provide details about the browser, server, cookies, and communication process.
- **Server headers may disclose technology information.** Exposing server software details can help attackers identify the technologies used by an application.
- **Cookies need proper security attributes.** The Secure flag restricts cookies to HTTPS connections, while HttpOnly prevents JavaScript from accessing them.
- **Improper response handling can lead to XSS.** User-generated content should be properly encoded or sanitized before being displayed.
- **CSP provides an additional layer of protection.** It restricts the sources from which scripts and other resources can be loaded.
- **HSTS helps prevent insecure connections.** It instructs browsers to use HTTPS when communicating with a website.
- **MIME-type sniffing can create security risks.** The `nosniff` directive instructs browsers to respect the declared Content-Type.
- **Improper redirects can expose users to malicious websites.** Redirect destinations should be validated to prevent open redirect vulnerabilities.

6. Security Recommendations

- Always use HTTPS to protect communication between browsers and web servers.
- Validate and sanitize user-controlled input, including URL paths, query parameters, and request body data.
- Implement proper authentication and authorization checks before allowing users to access or modify resources.
- Avoid placing passwords, session tokens, and other sensitive information in URLs.
- Configure cookies with appropriate Secure and HttpOnly attributes.
- Implement Content Security Policy to restrict untrusted content sources.
- Enable HSTS to enforce HTTPS connections.
- Configure X-Content-Type-Options with the `nosniff` directive.
- Use an appropriate Referrer-Policy to control information shared with external websites.
- Avoid unnecessarily exposing server software and version information through response headers.
- Properly encode user-generated content before displaying it in web pages.
- Validate redirect destinations to prevent open redirect vulnerabilities.
- Restrict unnecessary HTTP methods and ensure that every operation follows appropriate access-control rules.

7. Key Learnings

This room helped me build a clearer understanding of how web applications work beyond what we normally see in a browser. I learned how front-end technologies interact with back-end components and how information moves between a client and a web server through HTTP communication.

One of the most useful parts was understanding HTTP requests and responses in detail. Learning about request methods, headers, body formats, and status codes provides a foundation for inspecting web traffic and understanding how applications process user input.

I also gained a better understanding of HTTP security headers and their practical importance. Features such as CSP, HSTS, HttpOnly, Secure, and X-Content-Type-Options are not just configuration options; they help reduce the risk of common web application attacks.

The practical exercises involving GET, POST, and DELETE requests also helped connect the theoretical concepts with actual API operations.

My main takeaway: Before testing a web application for vulnerabilities, it is important to understand how it communicates, how it handles user input, and how it controls access to its resources. These fundamentals provide a strong starting point for learning web application penetration testing.

Status: Complete